INDEX

NAME: T. Srinivasa Reddy STD.: B.tech SEC.: 39 ROLL NO.: 1499 SUB.: observation.

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1		working with Numpy arrays.		
2.		working of pandas data frames		
3.		Basic Scenario Based programs using python for data science.		
4.		perform data processing on a given database		
5		Dataset splitting and cross validation using iris data.		
6		Linear regression using the Iris Data		
7.		Classification using iris dataset.		
8.		SVM Classification using iris dataset.		
9.		Implement K-means cluster using Iris Data set.		
10.		Hierarchial clustering using Iris Dataset.		
11.		Implement Random Forest and Analyze performance		
12.		Implement Boosting Techniques and Compare Results.		
13.		Implement a Basic Reinforcement learning models.		
14.		Build a Neural Network using Tensor Flow / Keras.		

Ex.No-1 Working with Numpy Arrays.

Aim:-

To write a Numpy arrays program to demonstrate the basic array concepts in jupyter Notebook.

Algorithm:-

1) Start the program
2) Import the Numpy library.
3) Define the 1-D array, 2-D array, 3-D array.
4) Print the memory address, the shape, the datatype and perform some basic operations.
5) Then create an array using built-in Numpy function and perform some mathematical functions.
6) print the output
7) stop the program.

Program:-

```
import numpy as np
Print(np.version)
```

output:-

1.21.5

o/p verified

Aim:

Q) Numpy program to print different types of arrays.

one-dimensional array.

Algorithm:-

1) start the process
2) Import the library:
3) call np.arrang(2,6) to create a sequence of integer values starting at 2 upto 6.
4) reshape (4) to structure the array into a 1-dimensional array
5) Store the resulting array into variable X.
6) print the value of x to the screen
7) End the process

code:-

```
import numpy as np
x = np.arange(0,6).reshape(4)
Print(x).
```

output:-

[2, 3, 4, 5]

three-dimensional array.

code:-

```
import numpy as np
x = np.arange(24).reshape(4,3,2)
Print(x)
```

output:-

```
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]
  [10 11]]

 [[12 13]
  [14 15]
  [16 17]]

 [[18 19]
  [20 21]
  [22 23]]]
```

O/p verified

3) Write a program to perform array slicing.

Algorithm:-

1. Start the process
2. Import the numpy library as np.
3. Create a 1D Numpy array [1, 2, 3, 4] using np.array() and store it in variable arr.
4. perform slicing operation 1 (arr[1:3:2]).
5. perform slicing operation 2 (arr[:3]).
6. perform slicing operation 3 (arr[::2]).
7. End the process.

Code:-

```
import numpy as np.
arr = np.array([1, 2, 3, 4])
print(arr[1:3:2])
print(arr[:3])
print(arr[::2])
```

output:-

```
[2]
[1 2 3]
[1 3]
```

O/p Verified

4) Numpy program to perform trigonometric functions

Algorithm:

1. Start the process
2. Import the numpy library as np.
3. Create a 1D Numpy array with values [0, 30, 60, 90] using np.array() and store it in variable arr.
4. calculate Sine pass arr into np.sin()
5. calculate cosine pass arr into np.cos()
6. Calculate Target pass arr into np.tan()
7. End the process.

Code:-

```
Import Numpy as np
arr = np.array([0,30,60,90])
Print(np.sin(arr))
Print(np.cos(arr))
Print(np.tan(arr))
```

atput:-

```
[0   -0.98803162  -0.30481062   0.89399805]
[1   0.15425145   -0.95241298   -0.44807362]
[0   -6.40533312  0.32004039    -1.99520041]
```

D/P verified

5

Aim:-

Numpy program to convert a list of numeric value into a one-dimensional Numpy array.

Algorithm:-

1. Start the process
2. Import the numpy library as np.
3. Create a standard python list l
4. output the contents of list l to the console.
5. pass List l to the np.array() function to convert it into a 1D Numpy array and store it in variable.
6. output the array a to the Console
7. End the process.

Code:-

```
import numpy as np.
l=[12.23, 13.32, 100, 36.32]
Print("original List", l)
    a= np.array(l)
Print("one-dim Numpy array:", a)
```

output:-

Original List:- [12.23, 13.32, 100, 36.32]

one-dim Numpy array:- [12.23 13.32 100. 36.32]

o/p Verified

6)

Ans:- Numpy program to create an array with values ranging from 12 to 38

Algorithm:-

1. Start the process.

Code:-

```
import numpy as
  np. x=
np. arange(12, 38)
Print(X)
```

output:-

[12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
29 30 31 32 33 34 35 36 37]

O/p Verified

7) Numpy Program to reverse an array (first element becomes last)

code:-

```
import numpy as
np import numpy
as np x=
np.a range (12, 38)

Print("original array")
  Print(X)
  Print("Reverse array")
  X = X[::-1]
    Print(X)
```

output:-

original array:-

[12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
28 29 30 31 32 33 34 35 36 37]

Reverse array:-

[37 36 35 34 33 32 31 30 29 28 27 26 25
24 23 22 21 20 19 18 17 16 17 16 15 14 13 12]

O/p verified

8) Numpy program to append values to the end of an array.

Code:-

```
import numpy as
    np x = [10, 20, 30]
    Print ("Original array")
    Print (x)
x = np. append (X[[40, 50, 60], [70, 80, 90]])
Print ("After append values to the end of the array")
    Print (x).
```

output:-

Original array:-

[10, 20, 30]

After append values to the end of the array.

[10 20 30 40 50 60 70 80 90]

Verified

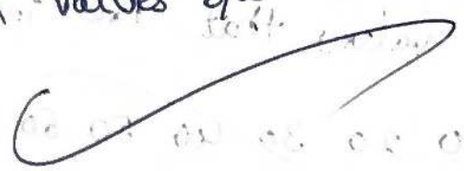
9) Numpy Program to find Common values b/w two arrays.

Code:-

```
import numpy as np
array 1 = np.array([0, 10, 20, 40, 60])
Print ("Array1:", array1)
  array 2 = [10, 30, 40]
Print ("Array 2:", array2)
Print ("Common values b/w two arrays:")
 Print (np. intersect 1d (array1, array2))
```

Output:-

```
Array 1: [0  10  20  40  60]
Array 2:  [10, 30, 40]
Common values b/w two arrays : [10  40]
```



O/p Verified

10) Numpy program to find the union of 2 arrays.

Code:-

```
import numpy as np
    array1 = np.array([0, 10, 20, 40, 60, 80])
    Print("Array1:", array1)
array2 = [10, 30, 40, 50, 70]
    Print("Array2:", array2)
Print("unique sorted array of values that are in either of the two input arrays:")
Print(np.union1d(array1, array2)
```

output:-

```
Array1 : [0 10 20 40 60 80]
Array2 : [10, 30, 40, 50, 70]
```

unique sorted array of values that are in either of the two input arrays: [0 10 20 30 40 50 60 70 80]

O/P verified

11) Numpy Program to concatenate two-dimensional arrays.

Code:-

```
import numpy as np
a = np.array ([[0,1,3],[5,7,9]])
b = np.array ([[0,2,4],[6,8,10]])
c = np.array ((a,b),1)
Print(c).
```

output:-

```
[[0 1 3 0 2 4]
 [5 7 9 6 8 10]]
```

O/P Verified

18) write a Numpy program to convert the values of centigrade degrees into fahrenheit degrees and vice versa

fahrenheit degree to centigrate degree:

Code:-

```
import numpy as np
fvalues = [0, 12, 45, 21, 34, 99.91, 32]
F = np.array(fvalues)
Print("values in fahrenheit degrees")
Print(F).
Print("values in centigrade degrees:")
Print(np.round((5* F/9 - 5*32/9).2))
```

output:-

values in fahrenheit degrees:

[10 12.45 21 34 99 91 32]

values in centigrade degrees:

[-17.78 -11.11 7.34 1.11 37.33 0]

centigrade degree to Fahrenheit degree.

Code:-

```
import numpy as np.
C values = [-17.78, -11.11, 7.34, 1.11, 37.73, 0]
c = np.array (c values)
Print (" values in centigrade degrees:")
Print(c).

Print (" values in fahrenhiet degrees :")
Print (np.round ((9*c/5+32), 2))
```

output-

values in centigrade degrees.

[-17.78 -11.11 7.34 1.11 37.73 0]

values in fahrenheit degrees.

[0 12 45. 21 34 99 91.32]

O/p verified

13) Numpy program to perform arithmetic operations

code:-

```
import numpy as np.
a = np.arrange(9, dtype=np.float64).reshape(3,3)
Print("First array:")
Print(a)
b = np.array([10, 20, 30])
Print("Second array:")
Print(b)
Print("Addition of arrays:")
Print(np.add(a,b))
Print("Subtraction of arrays:")
Print(np.subtract(a,b))
Print("Multiplication of arrays:")
Print(np.multiply(a,b))
Print("Division of arrays:")
Print(np.divide(a,b))
```

output:-

```
First array:
[[0, 1, 2] [3, 4, 5] [6, 7, 8]]
Second array
   [10  20  30]
addition of arrays ([10:21:32])
    [13, 24, 35]
    [16, 27, 38]
Subtraction of arrays: [[-10, -19, -28]]
    [-7, -16, -25] [-4, -13, -22]
multiplication of arrays.
    [[0, 20, 60] [30, 80, 150] [60, 140, 240]]
Division of arrays.
    [[0.0 0.05 0.06666666]] [0. 20. 20.
```

Exercise-2: Working of pandas Data frame

Aim:- To write a pandas program using sample data frame to perform operation in its dataframe.

Algorithm:-

① Open an IDE such as Jupyter notebook.

② import pandas packages.

③ Create a Dataframe using pandas and extract details of the dataset such as no.of datas in each column and renaming them.

④ perform operations on rows and columns and renaming them.

⑤ selecting, Adding and deleting datas on the dataset.

⑥ Handle the missing datas on the Dataframe.

① Pandas program to create and display a Dataframe from a specified dictionary data which has index labels.

Code:-

```
import pandas as pd.
import numpy as np.
exam_data = {'name': ['Anastasia', 'Dima', 'katherine'],
'score': [12.5, 9, 16.5];
'attempts': [1, 3, 2];
'qualify': ['yes', 'no', 'yes'] }
'labels' = ['a', 'b', 'c']
df = pd.DataFrame (exam_data, index = labels)
Print (df)
```

output:-

	name	score	attempts	qualify
a	Anastasia	12.5	1	Yes
b	Dima	9	3	no
c	katherine	16.5	2	yes.

(2) Pandas Program to display a Summary of the information about a specified Data frame, and it's data.

Code:-

```
df = pd Data frame (exam_data, index = labels)
Print("Summary of the basic information about Dataframe
and its data:")
Print(df info())
```

output:-

Summary of the basic information about this Dataframe and its data.

```
class 'Pandas-Core.frame Data frames)
Index: 3 entries, a to c
Data columns (total 4 columns):
0   name       10   non-null   object
1   score      8    non-null   float64
2   attempts   10   non-null   int64
3   qualify    10   non-null   object
dtypes float 64(1), int 64(1), object(2)
memory storage : 400 0 + byfes
```

None

O/p prompt

③ Pandas program to get the first 3 rows of a given data frame.

Code:-

```
df = pd.DataFrame(exam_data, index= labels)
Print("First three rows of the dataframe:")
Print(df.iloc[:3])
```

output:-

First three rows of the data frame:

	name	score	attempts	qualify
a	Anastisia	12.5	1	yes
b	Dima	9.0	3	No
c	Katherine	16.5	2	yes



4. Pandas program to select the 'name' and 'score' columns from the following data frame.

Code:-

```
df = pd.DataFrame(exam_data, index=labels)
Print("select specific columns:")
Print(df[['name', 'score']])
```

output:-

select specific columns:

	name	score
a	Anastasia	12.5
b	Dima	9.0
c	Katherine	16.5

O/p

Verified

(5) pandas program to select the rows where the no.of attempts in the examination is greater than 2

Code:-

```
df = pd.DataFrame(exam_data, index= labels)
Printf("Number of attempts in the examinator is greater than 2")
Print(df(df("attempts")>2))
```

Output:-

Number of attempts in the examinations is greater than 2

	name	score	attempts	quality
b	Dima	9.0	3	NO
d	James	Nan	3	No
f	Micheal	200	3	Yes.

O/p verified

6. pandas program to count the no.of rows. and columns of a data frame

Code:-

```
df = pd. Dataframe (exam_data, index=label)

total_rows = len (df. axes(0))
total_cols = len (df.axes(1))

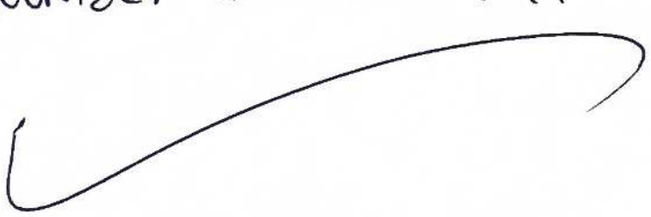
Print(" No.of Rows:" + str(total_rows))

Print(" Number of colcons :" str(total_cols))
```

output:-

Number of Rows : 10

Number of colums : 4.



O/P verified

(7) Pandas program to append a new row 'k' to Data frame with given values for each column. Now delete the new row and return the original data frame.

Code:-

```
df = pd.DataFrame(exam_data, index = labels)
Print ("original rows:")
  Print (df)
  Print("\n Append a new row:")
  df.loc['k'] = ('suresh', 15.5, 1, 'yes')
Printf(" Print all records after insert a new record:")
Print (dt).
Print("\n delete the new row and display the
original rows:") df=dt.
  df=df.drop('k')
Print(df).
```

output:-

original row:-

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Pima	9.0	3	no
c	Katherine	16.5	2	yes
d	James	Nan	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Mathew	14.5	3	yes
h	Laura	Nan	1	no.

@/pvnt

8) Pandas Program to select the rows where the score is missing ie, is Nan.

Code:-

```
df = pd.DataFrame (exam-data, index= labels)
Print("Rows where score is missing")
Print(df [df ["score"] is null (1))
```

output:-

Rows where score is Missing.

	name	score	attempts	qualify
d	James	Nan	3	no
h	Laura	Nan	1	no

(9) Pandas Program to sort the data frame first by 'name' in descending order, then by 'score' in ascending order.

Code:-

```
df = pd. Dataframe (exam.data, index = label)
Print ("original raws:")
    Print (df)
df. Sort_values (by = ('name', 'score'), ascending order:)
```

output:-

original raws:-

name	score	attempts	qualify.
Anastasia	12.5	1	yes.
Dima	9.0	3	No
Katherine	16.5	2	yes
James	Nan	3	no
Emily	9.0	2	no
Michael	20.0	3	yes
Matthew	14.5	1	yes
Laura	Nan	1	NO
Kevin	80	2	NO
Jonas	190	1	Yes.

O/p verified

(10) Pandas program to change the name 'James' to 'Suresh' in name column of the data frame.

code:-

```
df = pd. Data frame (exam_ data, index = labels)
  Print(" original rows:")
    Print (df)
  Print("In change the name 'James' to "shyam")
df ('name') = df('name') replace('James', 'shyam')
  Print (df).
```

output:-

original rows:-

name	score	attempts	qualify.
Anastasia	12.5	1	yes.
Dima	9.0	3	no
Katherine	16.5	2	yes.
James	Nan	3	no
Emily	9.0	2	NO
Michael	20.0	3	yes
Matthew	14.5	1	yes
Laura	Nan	1	no.
Kevin	8.0	2	no
Jonas	19.0	1	yes

Result:- Thus, the working of pandas dataframe was executed and verified successfully.

Exercise 3:

Basic scenario based programs for data science.

① Student Marks Analysis program.

Code:-

```
Marks = [78, 85, 92, 67, 88, 90, 76]

Print ("Highest mark", Max (marks))

Print ("lowest mark", Main (marks))

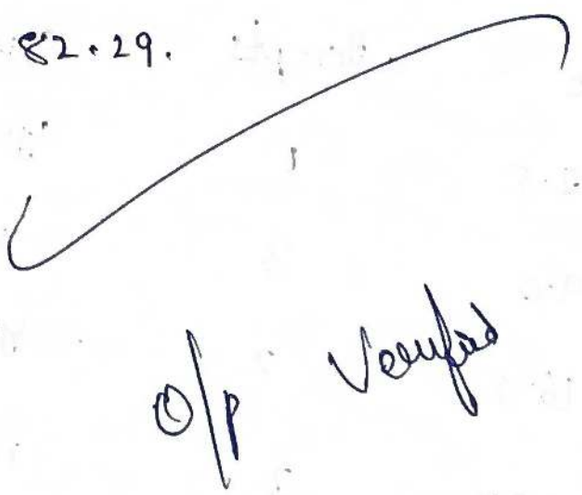
Print ("Average Mark", sum (Marks) / len (marks))
```

output:-

Highest mark : 92

lowest mark : 67.

Average Mark : 82.29.



O/P Verified

② Temperature Analysis program.

```
Temperature = (30, 32, 31, 29, 35, 34, 33)

Print ("Maximum Temperature :", Max (temperature))

Print ("Minimum Temperature :", Min (temperature))

Print ("Average Temperature :", Sum (temperature)/len (temperatu))
```

output:-

Maximum temperature : 35.

Minimum temperature : 29

Average temperature : 32.0

O/p Verified

③ weekly Sales Analysis Program.

```
Sales = (75.00, 3000, 2800, 3500, 4000, 3800, 4700)

Print ("Total Sales :", Sum (Sales))

Print ("Average Sales :", Sum (Sales)/len (Sales))
```

output:-

Total Sales : 23800

Average Sales : 34000.0

O/p Verified

④ Even and odd employee id's program.

```
ids = (101, 102, 103, 104, 105, 106)

even = []
odd = []

for i in ids:

    if i%2 == 0
    even.append(i)

Print("Even ids:", even)

print("odd ids:", odd)
```

output:- Even ids : (102, 104, 106)
odd ids : (101, 103, 105).

⑤ Count positive and Negative values.

```
Program values = [20, 5, 15, -10, -30, -2]

    positive = 0
    negative = 0
    for i in values:
        if i>=0:
            positive += 1
        elif i<0
            negative += 1
    Print("positive values:", positive)
    Print("negative values:", negative)
```

O/P verified

output:-
Positive values : 3
Negative values : 3.

⑥ Find. Replicate customer Ids Program.

code:-

```
customer_ids = [101, 102, 103, 102, 104, 105, 101]

    duplicates = []

    for i in customer_ids:

        if customer_ids.count(i) > 1 and not in duplicates.

        duplicates.append(i)

        Print("Duplicate Id's", duplicates)
```

output:-

Duplicate Ids : [101, 102].

⑦ product price analysis program.

Ans:-

```
Prices = (250, 450, 120, 600, 350)

Print("Highest price:", max(Prices))

Print("Lowest price:", min(Prices))
```

O/P

output:-

Highest price : 600

Lowest price : 120.

⑧ Employee salary increment program

```
Salary = (25000, 30000, 40000, 35000)
new_salary = []
for s in salary:
    new_salary.append(s*1.10)
Print("updated salaries:", new_salary)
```

output:-

updated salaries: (27500.0, 33000.0, 44000.0, 38500.0)

⑨ Attendence percentage program.

```
total_classes = 90
attended = 81
Percentage = (attended/total_classes)*100.
Print("Attendence percentage:", Percentage)
```

output:-

Attendence percentage: 90.0.

O/P

verified

⑩ students who passed exam.

```
Marks=(45, 67, 89, 32, 78, 55, 49)
Passed = []
for m in marks:
    if m >= 50
     passed.append(m)
Print("passed student marks", passed)
```

output:-

passed student marks: (67, 89, 78, 55]

⑪ Shopping Bill calculation program.

```
Prices = [120, 80, 150, 200]
total = sum (prices)
Print (" Total Bill", total)
```

output:-

Total Bill: 550.

Verified

(12) word frequency Analysis Program.

```
feedback = "good service good quality excellent service"
words = feedback.split()
for word in set(words)
print(word ":", words.count(word))
```

Sample output:-

good: 2
service: 2
quality: 1
excellent: 1

(13) Age group classification program.

```
ages = (12, 25, 45, 67, 15)
for age in ages.
if age < 18:
    print(age, "child")
elif age < 60:
    print(age, "senior citizen").
```

output:-

12 child
25 adult
45 adult
67 senior citizen
15 child.

O/P Verified

(14) Electricity bill calculation program.

```
units = 250
if units <= 100
    bill = units * 2
elif units <= 200:
    bill = 100*2 + 100*3 + (units - 200)*5
print("electricity bill:", bill)
```

output:-

Electricity bill : 750.

(15) Monthly Expense Analysis program.

```
expenses = (12000, 15000, 18000, 11000, 17000)
highest = max(expenses)
month = expenses.index(highest)+1
Print("Highest expense:", highest)
Print("Month:", month)
```

output:-

highest expense: 18000

month: 3.

O/P Verified

Exercise 4:

Perform data preprocessing on given dataset.

Aim:

To perform data preprocessing on a given dts.

Steps:-

① Loaded the iris dataset.

② Displayed the first five records.

③ checked the dataset structure using info().

④ checked for missing values (none were found in dataset).

⑤ Removed the id column

⑥ Converted the categorical species column into numerical values using label Encoder.

⑦ standardized the numerical features using standard Scaler

Program:-

```
# import libraries.
import pandas as pd
from sklearn preprocessing import label Encoder standard Scaler
# load dataset
df = pd.read_csv("Iris.csv")
# Display first 5 records.
Print("First 5 Records:")
Print(df.head())
# Display dataset information.
Print("\n Dataset information:")
Print(df.info())
# check for missing values.
Print("\n missing values:")
Print(df.isnull().sum())
# Remove the id column.
df = df.drop("Id", axis=1).
```

Encode the target column (species)

```
encoder = label/ Encoder()
df("species") = encoder.fit_transform(df("species"))
Print("\n Encoded species:")
Print(df.head())
```

feature scaling.

```
Scaler = standard scaler()
features = (" sepalength (cm"; "sepal width (cm)",)
            " petallength cm"; "petalwidth cm").
df(features): Scalar.fit_transfe (df(features))
Print("\n pre processor of Dataset:")
print(df.head())
```

Sample output:-

first 5 Records.

	id	sepal lengthcm	sepalwidth	petal length cm.	
0	1	5.1	3.5	1.4	0.2 Iris = setsa.
1	2	4.9	3.0	1.4	0.2
2	3	4.7	3.2	1.3	0.2
3	4	4.6	3.1	1.5	0.2
4	5	5.0	3.6	1.4	0.2

Dataset Information:-

<class 'pandas core frame Data frame'>

Range index: 150 entries 0 to 149

Data columns (total 6 columns.

id
sepal length cm
Sepal width cm
petal length cm
petal width cm
species.

Missing values:

id	0
sepal length cm	0
sepal width cm	0
petal length cm	0
petal width cm	0
species	0

Encoded species:-

	Sepal length cm	sepal width cm	petal length	cm	
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Preprocessed dataset (first 5 rows)

	Sepal length cm	sepal width cm	petal length cm	petal width cm	
0	-0.9007	1.0190	-1.3402	-1.3154	0
1	-1.1430	-0.1319	-1.3402	-1.3154	0
2	-1.3854	0.3284	-1.3407	-1.3154	0
3	-1.5065	0.0982	-1.2034	-1.3154	0
4	-1.0218	1.2492	-1.3402	-1.3154	0

Exercise No: 5

Dataset Splitting and Cross-Validation Using Iris Dataset

This program demonstrates 2 important techniques in machine Learning.

1) Data set splitting
2) K-Fold Cross-Validation.

Code:-

```
# Dataset splitting and Cross-Validation.
# Using Iris Dataset.

from sklearn.datasets import load_iris.
from sklearn.model_selection import train_test_split, KFold,
    cross_val_score.
from sklearn.linear_model import Logistic Regression.
from sklearn.metrices import accuracy_score.

iris = load_iris()
X = iris.data
y = iris.target.
Print("Dataset shape:", X.shape)
Print("Number of samples:", len(X))
Print("Number of Features:", X.shape[1])
Print("Target Classes:", iris.target_names)

X_Train, X_test, y_train, y_test = train_test_split(
                X,
                y,
                test_size = 0.20,
                random_state=42,
                startify = y
            )
```

```
Print("\n Dataset splitting")
Print("-----------")
Print("Training samples:", len(X_train))
Print("Testing samples :", len(X_test))

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
accuracy= accuracy_score(y_test, y_pred)
Print("\n Test set performance")
Print("----------")
Print("Test Accuracy:", round(accuracy, 4))
Kf = KFold(
        n_splits=5,
        shuffle=True,
        random_state=42
    )
CV_scores = cross_val_score(
                model,
                X,
                y,
                cv=kf,
                scoring='accuracy'
            )
Print("\n 5-fold cross-validation")
Print("-----------")
for i, score in enumerate(cv_scores, start=1):
        Print("fold", i, "Accuracy:", round(score, 4))
Print("\n cross-validation Results")
Print("----------")
Print("Average Accuracy:",
        round(cv_scores.mean(), 4))
Print("standard Deviation:",
        round(cv_scores.std(), 4))
```

Explanation of the program

Step 1 - Load Iris Dataset.

```
iris = load_iris()
```

The iris Dataset contains 150 flower samples belonging to 3 classes:

- setosa
- Versicolor
- Virginica.

Each sample contains four features:

1. sepal length
2. sepal width
3. petal length
4. petal width.

we separate the data into:

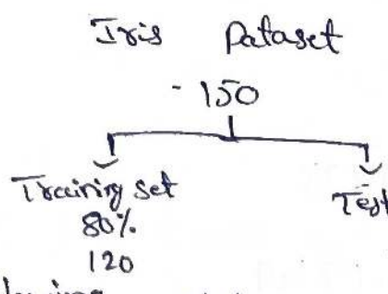
```
X = iris.data
Y = iris.target
```

Here:

- X → input features
- Y → target class

Dataset splitting

The dataset is divided into:



The following statement performs the split:

```
train_test_split(
    X,
    y,
    test_size = 0.20,
    random_state = 42,
    stratify = y
)
```

test_size = 0.20

20% of the data is used for testing.

Therefore:

150 x 20% = 30 testing samples.

and

150 - 30 = 120 training samples

random_state = 42

Ensures that the same random split can be reproduced.

Stratify = y:

Maintains approximately the same class distribution in both training and testing datasets.

Model Training

we use Logistic Regression:

model = LogisticRegression(max_iter=200)

The model is trained using:

model.fit(X_train, Y_train)

The model learns the relationship between the four flower measurements and the corresponding flower species

Testing the Model.

The trained model predicts the classes of unseen test samples.

Y_pred = model.predict(X_test)

The predictions are compared with the actual values:

accuracy_score(Y_test, Y_pred)

for example:

Actual	Predicted	
setosa	setosa	✓
versicolor	versicolor	✓
Virginica	virginica.	

K- fold cross validation.

Instead of depending only one train-test split,

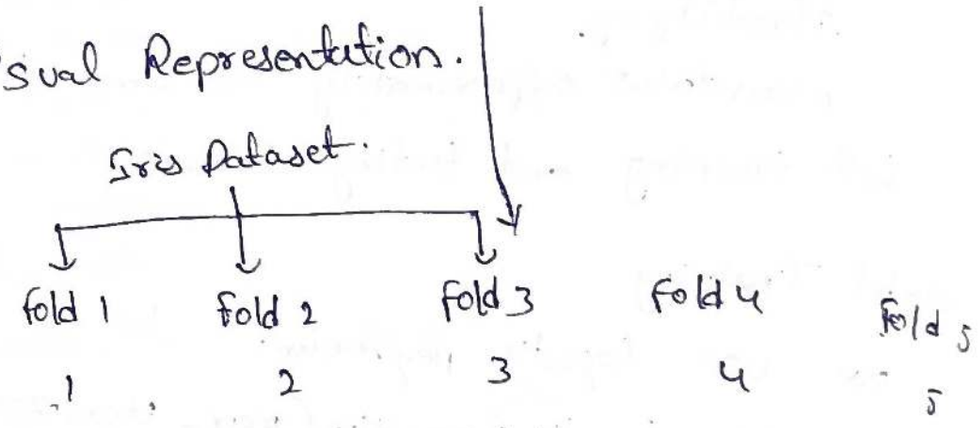
K-fold cross-validation divides the dataset into k parts called folds.

for this experiment:

n splits = 5.

therefore, we use 5-fold cross-validation

Visual Representation.



During each iteration:

Iteration 1:

Testing → fold 1

Training → Fold 2 + 3 + 4 + 5.

Iteration 2:

Testing

Ex no: 6 Linear Regression Using the Iris Dataset

Program:

```
# Linear Regression Using Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
iris = load_iris()
X = iris.data
feature_names = iris.feature_names
# predict: petal length
# column 2 = petal length
y = iris.data[:, 2]
# Use sepal length, sepal width and petal width
# as input features
X = iris.data[:, [0, 1, 3]]
print("Features:")
print(feature_names)
print("\n Dataset shape:", X.shape)
print("Target shape:", y.shape)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
print("\n Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
model = LinearRegression()
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)
Print("\n Actual vs predicted values")
Print("-----------------------")
for Actual, Predicted in zip(Y_test[:10], Y_pred[:10]):
    Print("Actual:", round(actual, 2),
          "predicted:", round(Predicted, 2))
mse = mean_squared_error(Y_test, Y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)
Print("\n Model Evaluation")
Print("-----------------------")
Print("Mean Squared Error (MSE):", round(mse, 4))
Print("Root mean Squared Error (RMSE):", round(rmse, 4))
Print("Mean Absolute Error (MAE):", round(mae, 4))
Print("R2 score:", round(r2, 4))
Print("\n Model coefficients")
Print("-----------------------")
for name, coefficient in zip(
    ["Sepal Length", "Sepal width", "petal width"],
                                    model.coef_):
    Print(name, ":", round(coefficient, 4))
Print("Intercept:", round(model.intercept_, 4))
```

Expected output :-

The exact numerical values can vary slightly depending on the train_test_split, but the output will be similar to:

Features:
['sepal length (cm)', 'sepal width (cm)'
'petal length (cm)', 'petal width (cm)']

Dataset shape: (150, 3)
Target shape: (150,)

Training samples : 120
Testing samples : 30

Actual vs predicted values

Actual: 5.1 predicted : 5.1
Actual : 4.0 predicted : 4.2
Actual : 1.3 predicted : 1.5
Actual : 1.5 predicted : 1.4
Actual : 4.7 predicted : 4.6

Model Evaluation

Mean squared Error (MSE) : 0.19
Root Mean squared Error (RMSE) : 0.44
Mean Absolute Error (MAE) : 0.32
R_2 score : 0.94

Model coefficients

sepal length : 0.72
sepal width : -0.45
Petal width : 1.35
Intercept : -1.02

Exp. no: 7. KNN classification using Iris Dataset.

Program:

```
# KNN classification using Iris Dataset.

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import standardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrices import(
                    accuracy_score,
                    confusion_matrix,
                    precision_score,
                    recall_score,
                    f1_score
                )
iris = load_iris()
X = iris.data
Y = iris.target
Print("Dataset shape:", X.shape)
Print("Number of samples:", len(X))
Print("Number of features:", X.shape[1])
Print("Target classes:", iris.target_names)

X_train, X_test, Y_train, Y_test = train_test_split(
                    X,
                    Y,
                    test_size = 0.20,
                    random_state=42,
                    stratify= Y
                )
```

```
Print("In Dataset Splitting")
Print("-----------")
Print("Training samples:", len(X_train))
Print("Testing samples:", len(X_test))

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighbourClassifier(n_neighbours=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
Precision = precision_score(
                y_test,
                y_pred,
                average='weighted'
            )
recall = recall_score(
                y_test,
                y_pred,
                average='weighted'
            )
f1 = f1_score(
                y_test,
                y_pred,
                average='weighted'
            )
Print("\nKNN Model Evaluation")
Print("-----------")
Print("Accuracy:", round(accuracy, 4))
Print("Precision:", round(precision, 4))
Print("Recall:", round(recall, 4))
Print("F1-Score:", round(f1, 4))
Print("\nConfusion Matrix")
Print("-----------")
Print(cm).
```

Output:-

Dataset shape: (150, 4)

Number of Samples : 150

Number of features : 4

Target Classes : ['setosa' 'versicolor' 'virg

Dataset splitting

Training samples: 120

Testing Samples: 30

KNN Model Evaluation

Accuracy : 1.0

Precision : 1.0

Recall : 1.0

F1-score : 1.0

Confusion Matrix

[[10 0 0]
 [0 10 0]
 [0 0 10]]

O/P verified

Ex. No: 5 SVM classification using Iris Dataset.

Program:

```
# SVM classification using Iris dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

iris = load_iris()
X = iris.data
Y = iris.target
print("Dataset shape:", X.shape)
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)
X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.20,
    random_state=42,
    stratify=Y
)
print("\n Dataset splitting")
print("-----")
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Scaler = StandardScaler()
X_train = Scaler.fit_transform(X_train)
X_test = Scaler.transform(X_test)

model = SVC(
        kernel='linear',
        C=1.0
    )
model.fit(X_train, Y_train)
Y_pred = model.predict(X_test)

accuracy = accuracy_score(Y_test, Y_pred)
cm = confusion_matrix(Y_test, Y_pred)

Precision = precision_score(
            Y_test,
            Y_pred,
            average='weighted'
        )
    recall = recall_score(
        y_test,
        y_pred,
        average='weighted'
    )
f1 = f1_score(
        y_test,
        y_pred,
        average='weighted'
    )

Print("\n SVM Model Evaluation")
Print("-----------------")
Print("Accuracy:", round(accuracy, 4))
Print("Precision:", round(Precision, 4))
Print("Recall:", round(recall, 4))
Print("F1-score:", round(f1, 4))
Print("\n Confusion Matrix")
Print("-----------------")
Print(cm)
```

output:-

Dataset shape: (150,4)
Number of samples : 150
Number of features : 4
Target classes : ['setosa' 'versicolor' 'virginica']

Dataset Splitting

Training samples : 120
Testing samples : 30

SVM model Evaluation

Accuracy: 1.0
Precision: 1.0
Recall : 1.0
F1-Score : 1.0

Confusion Matrix

[[10 0 0]
[0 10 0]
[0 0 10]]

0/p
Verified

Ex.no: 9 Implement K-Means clustering using Iris Dataset

Program:-

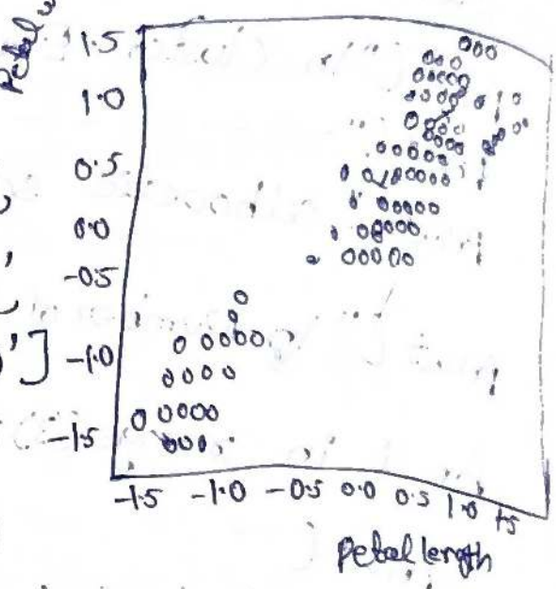
```
# K-Means clustering using Iris Dataset
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrices import silhouette_score
import matplotlib.pyplot as plt
iris = load_iris()
X = iris.data
Y = iris.target
Print("Dataset shape:", X.shape)
Print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
Print("In features:")
Print(iris.feature_names)
Scaler = StandardScaler()
X_scaled = Scaler.fit_transform(X)
KMeans = KMeans(
    n_clusters = 3,
    random_state = 42,
    n_init = 10
)
Kmeans.fit(X_scaled)
labels = Kmeans.labels_
centroids = Kmeans.cluster_centers_
```

```
Print("\n Cluster labels :")
Print(Labels)
Print("\n Cluster Centroids:")
 Print(Centroids)
Silhouette = silhouette_score(X_scaled, labels)
Print("\n Cluster Evaluation")
Print("----------------")
Print(" Silhouette score:", round(silhouette, 4))
Print("\n Number of Samples in each cluster:")
for i in range(3):
  Print(
      "Cluster", i,
      ":",
      sum(labels == i),
      "Samples"
    )
 plt.figure(figsize=(8,6))
  plt.scatter(
          X_scaled[:, 2],
          X_Scaled[:, 3],
          C=labels
      )
      plt.scatter(
          Centroids[:, 2],
          Centroids[:, 3],
          marker='X',
          s=200
      )
    plt.xlabel("petal length")
    plt.ylabel("petal width")
    plt.title("K-means clustering on Iris Dataset")
      plt.show()
```

Output:-

Dataset shape: (150, 4)
Number of samples: 150
Number of features: 4

Features:
['sepal length (cm)',
'sepal width (cm)',
'petal length (cm)',
'petal width (cm)']



cluster labels:
[1 1 1 1 1 1 1 1 1 1 1 . . .]

cluster centroids:

[[-0.99 0.87 -1.30 -1.25]
[1.13 0.09 0.98 1.01]
[-0.05 -0.88 0.35 0.30]]

cluster Evaluation
---

silhouette score: 0.4599

Number of samples in each cluster:

cluster 0: 50 samples
cluster 1: 50 samples
cluster 2: 50 samples.

O/p Verified

Ex.NO:-10 Hierarchial clustering Using Iris Dataset

```
# Hierarchial clustering Using Iris Dataset
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage

iris = load_iris()

X = iris.data
y = iris.target
Print("Dataset shape:", X.shape)
Print("Number of samples:", X.shape[0])
Print("Number of features:", X.shape[1])
Print("In features:")
Print(iris.feature_names)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
Linked = Linkage(X_scaled, method='ward')

Plt.figure(figsize=(10,6))

dendrogram(
        Linked,
        truncate_mode='lastp',
          p=20
        )
Plt.title("Hierarchial clustering Dendrogram")
Plt.xlabel("Data points/ clusters")
Plt.ylabel("Distance")
Plt.show()
```

```
model = AgglomerativeClustering(
        n_clusters = 3,
        linkage = 'ward'
    )
labels = model.fit_predict(X_scaled)
print("\n Cluster labels:")
print(labels)
silhouette = silhouette_score(
        X_scaled,
        labels
    )
print("\n Silhouette score:",
        round(silhouette, 4))
print("\n Number of samples in each cluster:")
for i in range(3):
    print(
        "cluster", i,
        ":",
        sum(labels == i),
        "samples"
    )
plt.figure(figsize=(8,6))
plt.scatter(
        X_scaled[:, 2],
        X_scaled[:, 3],
        c = labels,
    )
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.title("Hierarchial clustering on Iris dataset")
plt.show()
```

Output:

Dataset shape: (150, 4)

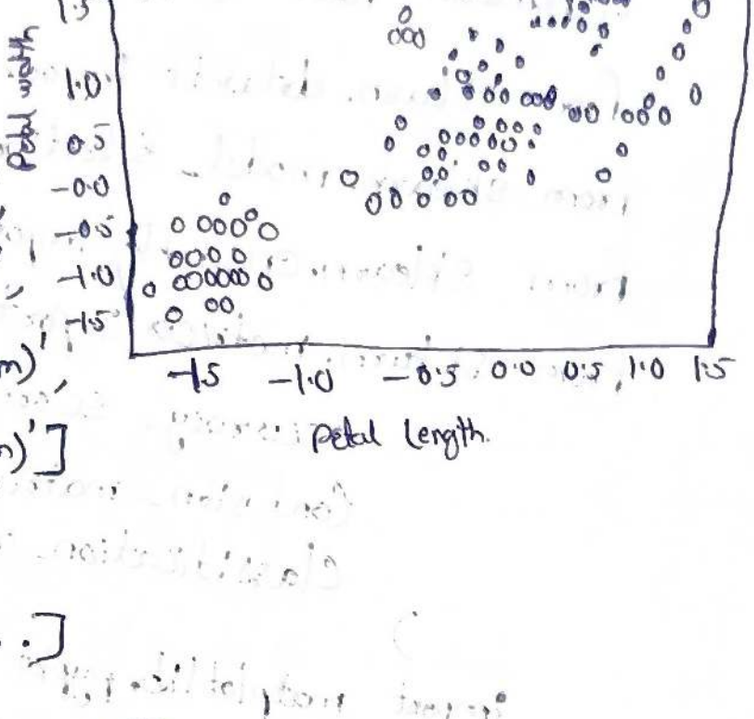
Number of samples: 150

Number of features: 4

Features:

['Sepal length (cm)',
'Sepal width (cm)',
'Petal length (cm)',
'Petal width (cm)']

Hierarchical clustering on Iris dataset



Cluster Labels:

[1 1 1 1 1 1 1 1 1 1 ...]

Silhouette score: 0.4467

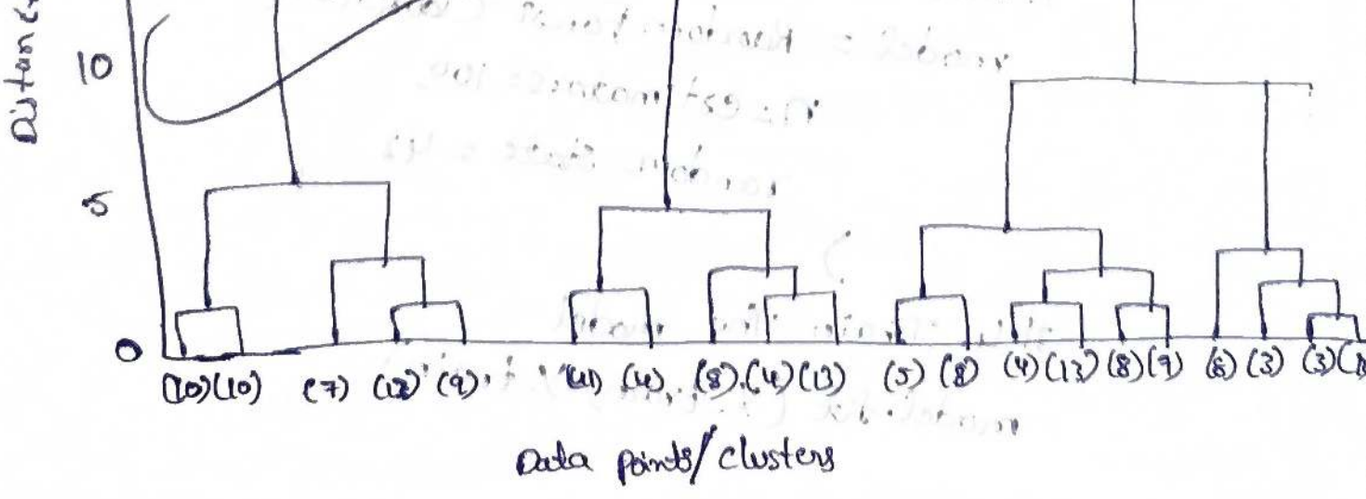
Number of samples in Each cluster:

Cluster 0: 64 samples

Cluster 1: 50 samples

Cluster 2: 36 samples

O/p



Ex.no: 11 Implement Random Forest and Analyze Performance

Program:

```
# Random Forest Classification Using Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
        accuracy_score,
        confusion_matrix,
        classification_report
    )
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
                                    X, y
                                    test_size = 0.20,
                                    random_state = 42,
                                    stratify = y
                                )

# 3. Create Random Forest model
model = RandomForestClassifier(
                n_estimators = 100,
                random_state = 42
            )

# 4. Train the model
model.fit(X_train, y_train)
```

```
#5. Make predictions.
y_pred = model.predict(x_test)
#6. Calculate accuracy.
accuracy = accuracy_score(y_test, y_pred)
print("Random forest classification")
print("----------")
Point("Number of trees:", 100)
Print("Accuracy:", accuracy)
#7. Confusion Matrix.
cm = Confusion_matrix(y_test, y_pred)
Print("In Confusion Matrix:")
Print(cm)
#8. Classification Report
Print("In Classification Report:")
Print(classification_report(
        x_test,
        y_pred,
        target_name = iris.target_names
    ))
#9. Display Confusion Matrix.
Plt.figure(figsize = (6,4))
sns.heatmap(
        cm,
        annot = True,
        fmt = "d",
        cmap = "Blues",
        xticklabels = iris.target_names,
        yticklabels = iris.target_names
    )
Plt.xlabel("predicted class")
Plt.ylabel("Actual class")
Plt.title("Random forest - Confusion Matrix")
plt.show()
```

Output:-

Random forest classification

Number of trees: 100

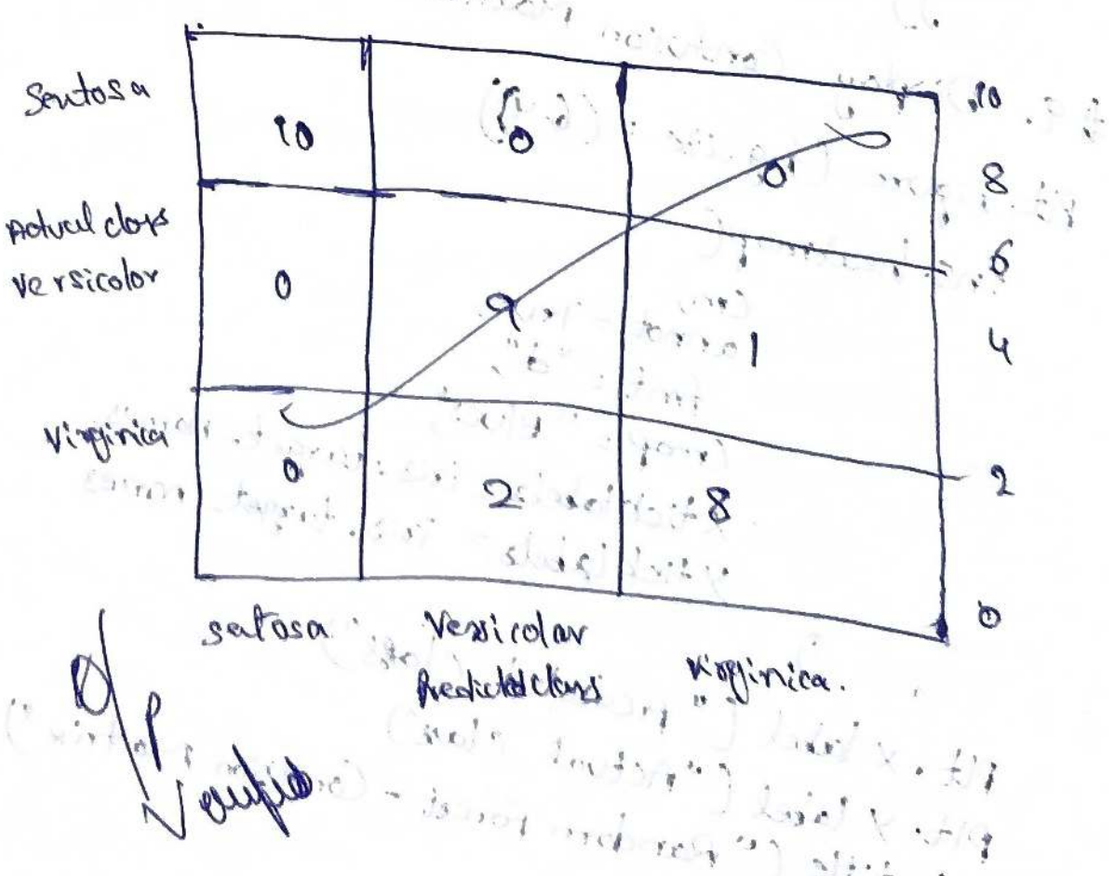
Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[10  0  0]
 [ 0 10  0]
 [ 0  1  9]]
```

classification Report:

	Precision	recall	f_1-score	Suppor
setosa	1.00	1.00	1.00	10
versicolor	0.91	1.00	0.95	10
Virginica	1.00	0.90	0.95	10



O/P
output

Ex.no:- 12 Implement Boosting Techniques and Compare Results.

Program:-

```
# implement Boosting Techniques and Compare Results
# Dataset Iris.
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostclassifier, Gradient Boosting classifier
from sklearn.metrics import accuracy_score, precision_score,
    recall score, f1_score.
import matplotlib.pyplot as plt.
iris = load_iris()
X = iris.data
Y = iris.target
X_train, X_test, Y_train, Y_test = train_test_split(
        X, Y,
        test_size = 0.20,
        random_state = 42,
        stratify = Y
    )
ada_model = AdaBoost classifier(
        n_estimators = 100,
        random_state = 42
    )
gb_model = Gradient Boosting classifier(
        n_estimators = 100,
        random_state = 42
ada_model.fit(X_train, Y_train)
gb_model.fit(X_train, Y_train)
ada_pred = ada_model.predict(X_test)
gb_pred = gb_model.predict(X_test)
```

```
def evaluate_model(name, y_test, y_pred):
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred, average="weighted"
    )
    recall = recall_score(
        y_test, y_pred, average="weighted"
    )
    f1 = f1_score(
        y_test, y_pred, average="weighted"
    )
    print("\n", name)
    print("-" * 30)
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1_score:", round(f1, 4))
    return accuracy, precision, recall, f1
ada_results = evaluate_model(
        "Ada Boost",
        y_test,
        ada_pred
    )
models = ["AdaBoost", "Gradient Boosting"]
accuracies = [
        ada_results[0],
        gb_results[0]
    ]
plt.figure(figsize=(7,5))
plt.bar(models, accuracies)
plt.xlabel("Boosting Technique")
plt.ylabel("Accuracy")
plt.title("Comparison of Boosting Techniques")
plt.ylim(0,1.1)
plt.show()
```

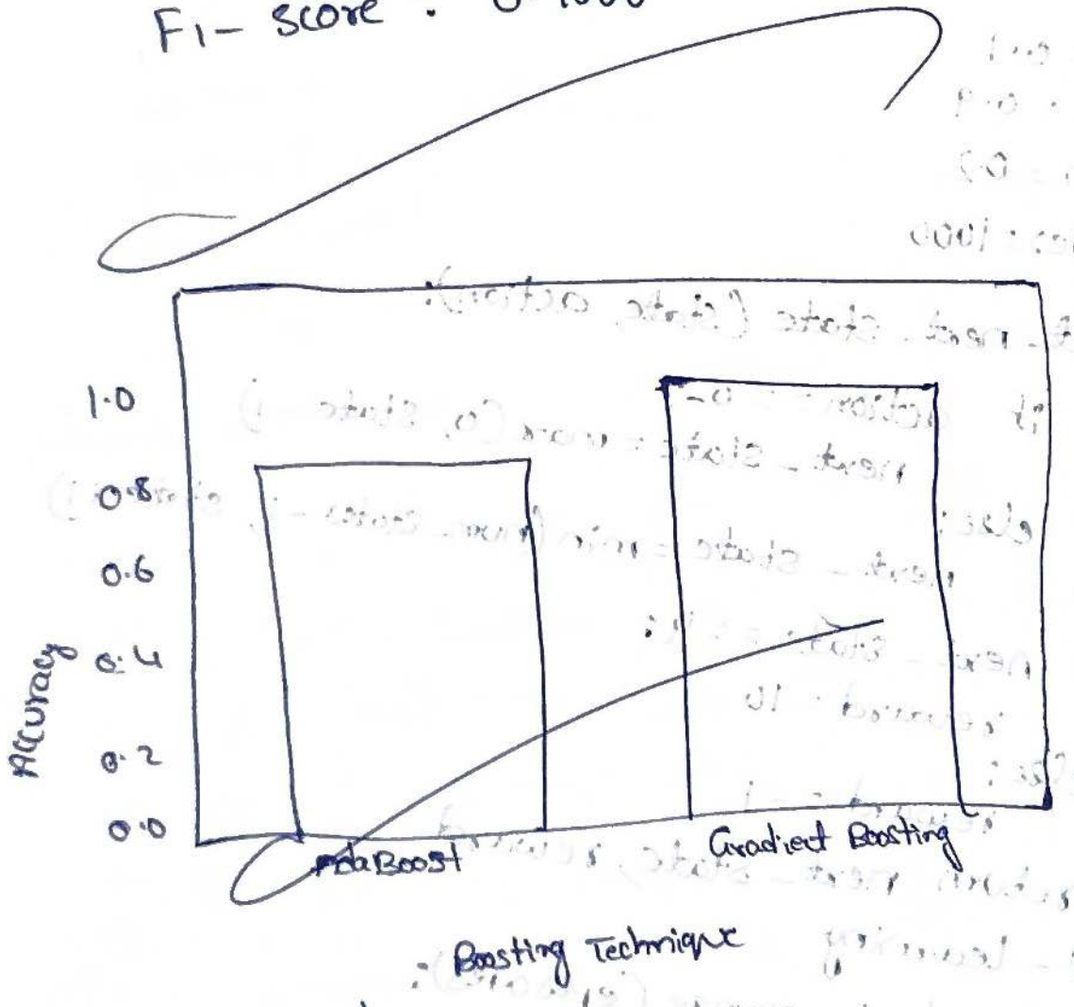
Output:-

Ada Boost

Accuracy : 0.9667
Precision : 0.9697
Recall : 0.9667
F1-score : 0.9666

Gradient Boosting

Accuracy : 0.9667
Precision : 0.9697
Recall : 0.9667
F1-score : 0.9666



O/P

Verified

Ex.no:- 13 Implement a Basic Reinforcement Learning

Program:-

```
import numpy as np
import random
# Number of states and actions
num_states = 5
num_actions = 2
# Actions
# 0 = left
# 1 = Right
# Intialite Q-table.
Q = np.zeros((num_states, num_actions))
# parametors
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 1000
def get_next_state (state, action):
    if action == 0 :
        next_state = max(0, state - 1)
    else:
        next_state = min(num_states - 1, state + 1)
    if next_state == 4:
        reward = 10
    else:
        reward = -1
    return next_state, reward
# Q-learning
for episode in range(episodes):
    state = 0
    while state != 4:
```

```
if random.random() < epsilon:
    action = random.randint(0, 1)
else:
    action = np.argmax(Q[state])
next_state, reward = get_next_state(state, action)
Q[state, action] = Q[state, action] + alpha * (
                    reward +
                    gamma * np.max(Q[next_state]) -
                      Q[state, action]
                    )
                state = next_state

# Display Q-table.
Print("Learned Q-Table:")
  Print(Q)

state = 0
path = [state]
while state != 4:
        action = np.argmax(Q[state])
next_state, reward = get_next_state(state, action)
      state = next_state
      path.append(state)
Print("\n Learned path:")
  Print(path)
Print("\n optimal policy:")
for state in range(num_states - 1):
    action = np.argmax(Q[state])
    if action == 0:
        Print("state", state, "->Left")
    else:
        Print("state", state, "->Right")
```

Output:

Learned Q-Table:

```
[[5.561  7.290]
 [5.561  8.100]
 [7.290  9.000]
 [8.100  10.000]
 [0.000  0.000]]
```

Learned path:

[0, 1, 2, 3, 4]

Optimal policy:

State 0 → Right
State 1 → Right
State 2 → Right
State 3 → Right.

O/P

Verified

Ex. no:- 14 Build a Neural Network Using Tensorflow/Keras

Program:-

```
# Build a Neutral Network Using Tensorflow/keras
# Dataset: Iris.

import numpy as np
import tensorflow as tf

from tensorflow.keras.models import sequential
from tensorflow.keras.layers import Dense.

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split(
                X, y,
                test_size = 0.20,
                random_state = 42,
                 stratify = y
               )

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

model= sequential([
    Dense(16, activation='relu', input_shape=(4,)),
    Dense(8, activation='relu'),
    Dense(3, activation='softmax')
    ])

model.Compile(
            optimizer = 'adam',
        loss = 'sparse_categorical_crossentropy',
        metrics= ['accuracy']
          )
```

```
model.summary()

history = model.fit(
        X_train,
        Y_train,
        epochs = 50,
        batch_size = 8,
        validation_split = 0.2,
        verbose = 1
    )

Loss, accuracy = model.evaluate(X_test, Y_test, verbose
Print("\n Test Loss:", round(loss, 4))
Print(" Test accuracy:", round(accuracy, 4))

Y_probability = model.predict(X_test, verbose=0)
Y_pred = np.argmax(Y_probability, axis=1)

Print("\n Classification Report:")
Print(classification_report(
        Y_test,
        Y_pred,
        target_names = iris.target_names
    ))
```

Output:-

Model: "sequential"

Layer (type)	Output shape	Param #
dense (Dense)	(None, 16)	80
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 3)	27

Total params : 243

Example:

Test Loss: 0.0865

Test Accuracy: 0.9667

Example classification result:

	Precision	recall	f_1-score
setosa	1.00	1.00	1.00
versicolor	0.91	1.00	0.95
virginica	1.00	0.90	0.95

o/p verified